Quiz 3 · Git & GitHub for collaboration

Friday, June 5, 2026

Today: Friday June 5

Before class

- **Perusall Assignment**: *DC* §9.1–9.2

Plan for today

- **Quiz 3** (first 20 minutes)
- Why version control exists, and what it buys you
- Git vs. GitHub, and the one workflow loop you'll use daily
- A live demo (watch, don't follow along yet)
- Your weekend mission: get Git working + a team repo

Helpful links

- Syllabus · AI policy · Schedule · Happy Git with R · *DC* Ch. 9 · GitHub

Quiz 3

Quiz 3: 20 minutes

- Closed-book, closed-notes, closed-internet, closed-AI.
- Covers **weeks 1–3**: data frames and ggplot2, plus this week's wrangling, the six verbs (`filter`, `select`, `mutate`, `arrange`, `group_by`, `summarise`) and grouped summaries.
- Pivoting appears **only at the concept level** (wide vs. narrow); no `pivot_*()` syntax required.
- Five short questions. Show your work.

When you finish, turn in your quiz up front and we'll start on Git.

Why version control?

You have already invented version control

Be honest, you've done some version of this:

```
analysis.R
analysis_v2.R
analysis_final.R
analysis_final_REAL.R
analysis_final_REAL_usethis_one.R
```

It sort of works, until you can't remember which is which, or two people email versions back and forth and the changes get lost.

> Tip
>
> **Version control** is the tool built for exactly this problem. *DC* §9.1 calls it a time machine for your project.

What version control actually gives you

From *DC* §9.1, a version control system lets you:

- **Record the full history** of every file, not just the latest copy
- **Revert** to any earlier state if something breaks
- **Manage contributions** from several people on one project
- **Draft freely** without touching the "good" version until you're ready

Instead of saving whole copies, Git stores the **incremental changes**, which is far more efficient and lets it rebuild any past state on demand.

It's not just for teams

DC §9.1.1: most version control is "self-collaboration."

- Working across **two computers** (laptop + lab machine)
- Coming back to a project **months later** and needing its history
- An **insurance policy** if your laptop is lost or dies

> **Note**
>
> And §9.1.2: paired with **literate programming** (your Quarto documents from week 1), version control is what makes an analysis *reproducible*, a complete, transparent record from raw data to final result.

Git vs. GitHub

Two different things

People say "git" for everything, but there are two pieces (*DC* §9.2):

Table 1: Git is the tool; GitHub is one place to keep it.

	Git	**GitHub**
What	The version-control *tool*	A *website* that hosts repos
Where	On your computer (local)	In the cloud (remote)
Job	Tracks changes, makes commits	Stores & shares the project

A **repo** (repository) is just a folder Git is watching, your `.qmd` files, data, images, all of it. You keep a **local** copy on your computer and a **remote** copy on GitHub, and you sync between them.

The daily loop

The four moves you'll actually use

Ninety percent of Git, every day, is this loop (*DC* §9.2.2):

1. **Edit & save** your files, as you already do.
2. **Stage + Commit** — take a snapshot, with a short message saying what changed.
3. **Pull** — grab any changes from GitHub *before* you send yours.
4. **Push** — send your commits up to GitHub.

> **Tip**
>
> **Commit early and often.** Each commit is a labeled stop on the time machine. Lots of small, well-described commits beat one giant mystery commit.

Write commit messages your future self can read

The message is how you (and your teammates) navigate the history later.

```
# Helpful
"Add body-mass-by-species summary table"
"Fix na.rm bug in flipper-length means"

# Useless
"stuff"
"asdf"
"changes"
```

> **Note**
>
> A commit with a vague message is a stop on the time machine with no label, you'll have no idea what's there without opening it.

Where this lives: the RStudio Git tab

Once Git is configured, you barely leave RStudio (*DC* §9.2, Fig. 9.2). A **Git** tab appears in the top-right pane with:

- A list of changed files, each with a **checkbox** to *stage* it
- A **Commit** button (opens a box for your message)
- **Pull** (down arrow) and **Push** (up arrow) buttons

That's the whole interface for the daily loop, no command line required for now.

Live demo

Watch this once

I'll run the full loop on the projector. **Don't follow along yet**, just watch the shape of it; you'll do it yourself this weekend.

1. Create a new repo on **GitHub** (with a README).
2. In RStudio: **New Project → Version Control → Git**, paste the repo URL.
3. Edit the README, **save**.
4. Git tab: **stage** the change → **Commit** with a message → **Push**.

5. Refresh the GitHub page, the change is there.
6. Edit a file *on GitHub*, then **Pull** in RStudio to bring it down.

> **Tip**
>
> Notice there was nothing about data analysis here. Git sits *underneath* the work you already do, it just snapshots and syncs the files.

How a team uses it

The remote is the shared hub (*DC* §9.2, Fig. 9.4). For your project team:

- One shared repo on GitHub; everyone has a local clone.
- Each person edits, commits, and pushes their work.
- **Always pull before you push**, so you start from everyone's latest.

```
        GitHub (shared remote)
         ↑ push        ↓ pull

you      teammate 1    teammate 2
```

When it goes wrong

The three you'll hit first

Everyone trips on these; there's a fix for each (*DC* §9.2.3):

Table 2: Common snags and their fixes.

Symptom	Cause	Fix
Merge conflict	Two people changed the same lines	Git asks *you* to pick; pull/push often to avoid
"My changes aren't tracked!"	Wrong RStudio **Project** open	Switch to the Project tied to that repo
Push rejected	A teammate pushed first	**Pull**, then push again
File-too-big warning (>50 MB)	Committing a big data file	Don't commit it; add to `.gitignore`

> **Warning**
>
> When truly stuck, Happy Git's troubleshooting chapter and the "Burn it all down" escape hatch (re-clone a fresh copy) will save you. Getting stuck is normal, not a sign you did it wrong.

What we covered today

- **Why:** version control = full history + revert + safe collaboration; a reproducibility backbone
- **Git vs. GitHub:** the local tool vs. the remote host; a repo is a tracked folder
- **The daily loop:** stage → commit (good message!) → pull → push, mostly from the RStudio Git tab
- **Teams:** one shared remote, everyone clones, *pull before you push*
- **Snags:** merge conflicts, wrong Project, rejected pushes, big files, each has a known fix

Your weekend

The mission

The in-class demo was the map. This weekend you walk the trail.

- Setup guide / Activity Canvas Link

Solo: follow Happy Git with R to register on GitHub, install Git, connect it to RStudio, and prove it works with a test repo.

Team: one person creates the project repo and adds the others as collaborators; everyone clones it and pushes at least one commit.

> **Tip**
>
> The setup is the genuinely fiddly part, so start early and use office hours. Once it works, it tends to keep working.

Wrap-up & looking ahead

Monday Jun 8: bring your configured laptop. We'll do a collaborative round-trip together, push, pull, and resolve a merge conflict on purpose, so the project workflow is second nature before you rely on it.

Upcoming Deadlines

- **Tonight, Fri 6/5, 11:59 p.m.**: HW 3
- **By Mon 6/8, class**: team repo working, **all members able to push** (today's setup guide)
- **Mon 6/8, in class**: bring your configured laptop

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · Happy Git with R · *DC* Ch. 9 · GitHub